

Research Gap Analysis

Paper 1: IaC Generation with LLMs: An Error Taxonomy and A Study on Configuration Knowledge Injection

The paper tests baseline LLMs, Naive RAG, and Graph RAG (with GR-OptMatch, GR-LLMSum, and GR-Ref enhancements) for generating AWS Terraform code inside a safe LocalStack emulation environment. It pushes technical validation success from 37.2% to 84.2% and overall success from 27.1% to 62.6%, but intent validation stalls at 72%-76% because of a "Correctness-Congruence Gap" — code can be technically correct yet not match what the user meant — and the full-reference GR-Ref approach overloads simple prompts. Because everything is scoped to single-provider AWS Terraform, the resulting research gap is clear: current knowledge-injected RAG frameworks can't bridge high-level architectural intent or adapt context retrieval across different cloud providers. The suggested fix is a dynamic, adaptive context-pruning system combined with multi-agent reasoning that adjusts retrieval depth to prompt complexity across multi-cloud setups.

Paper 2: GENSIAC: Toward Security-Aware Infrastructure-as-Code Generation with Large Language Models

GENSIAC fine-tunes CodeLlama 7B/13B with LoRA and 8-bit quantization on instruction-code pairs built from the GLITCH static analysis tool, raising security weakness detection F1 from ~0.30 to 0.858 (beating GPT-4) with near-perfect syntactic and functional correctness, in under 10 hours of training per epoch. Its limitations — missing rules like CWE-276, weaker Ansible transfer versus Chef/Puppet, and no live runtime validation — all stem from one root cause: the model only ever learns from what static scanners already flag. That creates the paper's core research gap, incomplete vulnerability coverage and no automated runtime cloud verification, and points toward the proposed improvement of feeding dynamic cloud sandbox feedback into the fine-tuning data so the model learns from real deployment behavior, not just static patterns.

Paper 3: INFRAFIX: Technology-Agnostic Repair of Infrastructure as Code

INFRAFIX repairs code across Ansible, Puppet, Chef, and Terraform simultaneously using an extended intermediate representation, a normalization database, and an SMT-based repair engine, hitting a 95.7% success rate across 254,288 scenarios. It breaks down exactly where static analysis runs out: unresolvable or uninstantiated resource names, shared-variable side effects, and non-filesystem resources that need custom state modules. The research gap this exposes is that symbolic, constraint-driven repair simply cannot reason about runtime variables or complex multi-resource cloud dependencies without manual rules — so the proposed improvement is to pair the SMT solver with an LLM-driven reasoning layer that can resolve those runtime values and dependency graphs dynamically.

Paper 4: GenKubeSec: LLM-Based Kubernetes Misconfiguration Detection, Localization, Reasoning, and Remediation

GenKubeSec combines a CodeT5+ detection model with a Mistral-7B remediation model to reach 0.990 precision and 0.999 recall on Kubernetes misconfigurations, with 100% expert-validated fixes and fast adaptation from just 1-5 examples. Its limitations are bounded by scope: CodeT5+'s 512-token limit misses 17% of longer manifests, near-identical rules like RBAC trigger false positives, and testing covered a small qualitative sample. That leaves a research gap around restrictive context limits and a lack of integration with broader cloud services beyond Kubernetes (AWS EC2, S3, IAM), which the paper suggests closing by upgrading to a long-context transformer (8k+ tokens) and expanding the misconfiguration taxonomy to cover full-stack AWS architectures.

Paper 5: Practical Integration of Large Language Models into Enterprise CI/CD Pipelines for Security Policy Validation: An Industry-Focused Evaluation

This framework layers an LLM Ensemble (GPT-4, fine-tuned models, ReAct filtering) on top of static scanners (OPA/Checkov) inside enterprise CI/CD pipelines, reaching $F1 = 0.95$ at 3.1 seconds latency, cutting prompt-injection success from 60% to 7%-12%, and reducing manual review time by 60%. The catch is cost: the ensemble needs roughly 3x the compute, still hallucinates on about 23% of ambiguous cases, and has only been validated on mid-scale, largely synthetic workloads. That defines the research gap — real-time LLM validation in CI/CD is still too expensive and hallucination-prone — with the proposed fix being to swap the costly ensemble for lightweight, locally fine-tuned models paired with retrieval-augmented policy-as-code engines to cut both cost and false alerts.

Overall Research Analysis

Across all five papers, the recurring research gap is the same at every level: each solution excels within one narrow slice of the IaC world — one cloud provider, one scripting language, or one platform like Kubernetes — but none validate a real, integrated multi-service cloud architecture end-to-end. Layered on top of that is a shared set of practical constraints (limited context windows, high compute cost, and hallucination rates near 23%) and a deeper conceptual gap, the "Correctness-Congruence Gap," where configurations pass technical checks without truly satisfying the user's intent or live security requirements. Closing these gaps calls for a unified, multi-service, cost-aware framework that combines adaptive retrieval, runtime feedback, and real-time cloud verification rather than another narrowly-scoped, single-domain solution.